

SOLIDWORKS DimXpert API (.NET) — Examples

(7/7): Get DimXpert Tolerance Example_VB-

SolidWorks.Interop.swdimxpert

SOLIDWORKS API Documentation (converted from CHM)

Get DimXpert Tolerance Example (VBA)

This example shows how to build a part and get attributes for the following DimXpert annotations:

- * Counterbore dimension tolerance
- * Depth dimension tolerance

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\face_plate_ads_geo.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Click Auto Dimension Scheme on the DimXpert toolbar.  
' 4. Ensure that all feature filters are selected.  
' 5. Click the green check mark to accept the settings.  
' 6. In the DimXpertManager tab of the Management Panel,  
'    expand Hole Pattern8 in the DimXpertManager tab of the  
'    Management Panel.  
' 7. Observe the following DimXpert annotations: Counterbore3 and Depth3  
' 8. Open an Immediate window.  
' 9. Ensure that the latest SOLIDWORKS DimXpert type library is loaded  
'    in Tools > References.  
' 10. Ensure that the Microsoft Scripting Runtime library is loaded  
'    in Tools > References.  
  
'  
' Postconditions:  
' 1. Writes the output to c:\temp\dimXpertInfo.txt.  
' 2. Inspect the Immediate Window.  
'  
' NOTE: Because these parts are used elsewhere, do not save changes.  
' -----
```

Option Explicit

Dim strs As New Collection

Dim dimXpertPart As SwDimXpert.dimXpertPart

Sub Main()

Dim swapp As SldWorks.SldWorks

Set swapp = Application.SldWorks

Dim swModelDoc As SldWorks.ModelDoc2

Set swModelDoc = swapp.**ActiveDoc**

If swModelDoc Is Nothing Then

Exit Sub

End If

Dim f As New FileSystemObject

```

Dim textStr As TextStream
Set textStr = f.CreateTextFile("C:\temp\dimXpertInfo.txt", True)
If textStr Is Nothing Then
    Debug.Print "Error creating temp file."
    Exit Sub
End If

Call log("Starting DimXpert log...", textStr)
Call retrieve_info_text(swapp, textStr)
textStr.Close

End Sub

Private Sub log(text As String, textStr As TextStream)
    Debug.Print text
    textStr.WriteLine (text)
End Sub

Private Sub retrieve_info_text(swapp As SldWorks.SldWorks, textStr As TextStream)
    Dim dimXpertMgr As SldWorks.DimXpertManager

    Set dimXpertMgr =
swapp.IActiveDoc2.Extension.DimXpertManager(swapp.IActiveDoc2.IGetActiveConfiguration().Name,
True)

    Call log("Model: " & swapp.IActiveDoc2.GetPathName, textStr)
    Dim dimXpertPartObj As dimXpertPart
    Set dimXpertPartObj = dimXpertMgr.dimXpertPart

    Set dimXpertPart = dimXpertPartObj

    Dim vAnnotations As Variant

    vAnnotations = dimXpertPart.GetAnnotations()

    Call log("-----", textStr)
    Call log("Annotations...", textStr)
    Call log("-----", textStr)

    Dim annotationTemp As DimXpertAnnotation
    Dim annotationIndex As Long
    For annotationIndex = 0 To UBound(vAnnotations)
        Set annotationTemp = vAnnotations(annotationIndex)

        Dim AnnotationDataText As Collection

        Set AnnotationDataText = AnnotationData(annotationTemp)

```

```

    Dim annotationTextIndex As Long
    For annotationTextIndex = 1 To AnnotationDataText.Count
        Call log(AnnotationDataText(annotationTextIndex), textStr)
    Next
Next

```

End Sub

```
Public Function AnnotationData(annotation As DimXpertAnnotation) As Collection
```

```
    Dim annoType As Long
```

```
    'general info
```

```
    Call GeneralInfo(annotation)
```

```
    annoType = annotation.Type
```

```
    If annoType = swDimXpertDatum Then
```

```
        Call DatumData(annotation)
```

```
    ElseIf annoType = swDimXpertGeoTol_Position Then
```

```
        Call PositionData(annotation)
```

```
    ElseIf annoType = swDimXpertGeoTol_CompositePosition Then
```

```
        Call CompositePositionData(annotation)
```

```
    ElseIf annoType = swDimXpertGeoTol_Symmetry Then
```

```
        Call SymmetryData(annotation)
```

```
    ElseIf annoType = swDimXpertGeoTol_Concentricity Then
```

```
        Call ConcentricityData(annotation)
```

```
    ElseIf annoType = swDimXpertGeoTol_LineProfile Then
```

```
        Call LineProfileData(annotation)
```

```
    ElseIf annoType = swDimXpertGeoTol_CompositeLineProfile Then
```

```
        Call CompositeLineProfileData(annotation)
```

```
    ElseIf annoType = swDimXpertGeoTol_SurfaceProfile Then
```

```
        Call SurfaceProfileData(annotation)
```

```
    ElseIf annoType = swDimXpertGeoTol_CompositeSurfaceProfile Then
```

```
        Call CompositeSurfaceProfileData(annotation)
```

```
    ElseIf annoType = swDimXpertGeoTol_Angularity Or annoType = swDimXpertGeoTol_Parallelism
Or annoType = swDimXpertGeoTol_Perpendicularity Then
```

```
        Call OrientationData(annotation)
```

```

ElseIf annoType = swDimXpertGeoTol_TotalRunout Then
    Call TotalRunoutData(annotation)

ElseIf annoType = swDimXpertGeoTol_CircularRunout Then
    Call CircularRunoutData(annotation)

ElseIf annoType = swDimXpertGeoTol_Flatness Then
    Call FlatnessData(annotation)

ElseIf annoType = swDimXpertGeoTol_Circularity Then
    Call CircularityData(annotation)

ElseIf annoType = swDimXpertGeoTol_Cylindricity Then
    Call CylindricityData(annotation)

ElseIf annoType = swDimXpertGeoTol_Straightness Then
    Call StraightnessData(annotation)
ElseIf annoType = swDimXpertGeoTol_Tangency Then
    Call TangencyData(annotation)
Else ' any of the dimension tolerance types
    Call DimensionToleranceData(annotation)

End If

```

```

Set AnnotationData = strs

```

```

End Function

```

```

Private Sub Clear(strs As Collection)

```

```

    Dim n As Long

```

```

    strs.Remove (strs.Count)

```

```

    If Not strs.Count = 0 Then

```

```

        Call Clear(strs)

```

```

    End If

```

```

End Sub

```

```

Private Sub GeneralInfo(annotation As DimXpertAnnotation)

```

```

    Dim annoType As String

```

```

    Dim modelObj As Object

```

```

    Dim modelFeature As SldWorks.feature

```

```

    If Not strs.Count = 0 Then

```

```

        Call Clear(strs)

```

```

    End If

```

```

    strs.Add ("")

```

```

    strs.Add ("Name: " + annotation.Name)

```

```

annoType = annotationTypeNameFromObject(annotation)
strs.Add ("Type: " + annoType)
strs.Add ("Display Entity: " + DisplayEntity(annotation))

Set modelObj = annotation.GetModelFeature
Set modelFeature = modelObj
If Not (modelFeature Is Nothing) Then
    strs.Add ("ModelFeature: " + modelFeature.Name + " (" + modelFeature.GetTypeName2() +
    ")")
End If
End Sub

Private Sub DatumData(annotation As DimXpertDatum)
    ' the datum letter
    strs.Add ("")
    strs.Add ("Datum Letter: " + annotation.Identifier)

End Sub

Private Sub PositionData(annotation As DimXpertPositionTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean, value As Double
    Dim boolstatus As Boolean

    strs.Add ("")
    strs.Add ("Position Tolerance Compartment:")

    ' the shape of the tolerance zone
    strs.Add (" Zone Type: " + PositionZoneType(annotation.zoneType))

    ' the zone vector if the tolerance zone is planar
    If annotation.zoneType = swDimXpertPositionZoneType_PlanarPosition Then
        boolstatus = annotation.GetPlanarZoneVector(I, J, K)
        strs.Add (" Direction: " + FormatVector(I, J, K))
    End If

    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

    ' the material condition modifier applied to feature
    strs.Add (" Modifier: " + mcmStr(annotation.modifier))

    ' the projected tolerance zone when specified
    boolstatus = annotation.GetProjectedZone(enabled, value)
    Call FormatProjectedZone(enabled, value)

    ' the datum reference frame

```

```
Call DatumsStr(annotation)
```

```
End Sub
```

```
Private Sub CompositePositionData(annotation As DimXpertCompositePositionTolerance)
```

```
Dim I As Double, J As Double, K As Double
```

```
Dim enabled As Boolean, value As Double
```

```
Dim boolstatus As Boolean
```

```
strs.Add ("")
```

```
strs.Add ("Composite Position Tolerance Compartment")
```

```
' the shape of the tolerance zone
```

```
strs.Add (" Zone Type: " + PositionZoneType(annotation.zoneType))
```

```
' the zone vector when the zone is planar
```

```
If annotation.zoneType = swDimXpertPositionZoneType_PlanarPosition Then
```

```
    boolstatus = annotation.GetPlanarZoneVector(I, J, K)
```

```
    strs.Add (" Direction: " + FormatVector(I, J, K))
```

```
End If
```

```
' the tolerance value
```

```
strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
```

```
' the material condition modifier
```

```
strs.Add (" Modifier: " + mcmStr(annotation.modifier))
```

```
' the projected tolerance zone when specified
```

```
boolstatus = annotation.GetProjectedZone(enabled, value)
```

```
Call FormatProjectedZone(enabled, value)
```

```
' the datum reference frame for the pattern location
```

```
Call DatumsStr(annotation)
```

```
' the datum reference frame for the feature to feature location
```

```
strs.Add ("Composite datums:")
```

```
strs.Add (" Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
```

```
strs.Add (" Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True", "False"))
```

```
strs.Add (" Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True", "False"))
```

```
End Sub
```

```
Private Sub SymmetryData(annotation As DimXpertSymmetryTolerance)
```

```
Dim I As Double, J As Double, K As Double
```

```
Dim boolstatus As Boolean
```

```
strs.Add ("")
```

```

strs.Add ("Symmetry Tolerance Compartment")

' the tolerance value
strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

' the material condition modifier applied to feature
strs.Add (" Modifier: " + mcmStr(annotation.modifier))

' the datum reference frame
strs.Add ("")
Call DatumsStr(annotation)
' the direction of the planar zone
strs.Add ("")
boolstatus = annotation.GetZoneDirection(I, J, K)
strs.Add ("Planar Zone Direction: " + FormatVector(I, J, K))

```

End Sub

```
Private Sub ConcentricityData(annotation As DimXpertConcentricityTolerance)
```

```

strs.Add ("")
strs.Add ("Concentricity Tolerance Compartment")

' the tolerance value
strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

' the material condition modifier applied to feature
strs.Add (" Modifier: " + mcmStr(annotation.modifier))

' the datum reference frame
Call DatumsStr(annotation)

```

End Sub

```
Private Sub TotalRunoutData(annotation As DimXpertTolerance)
```

```

strs.Add ("")
strs.Add ("Total Runout Tolerance Compartment")

' the tolerance value
strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

' the datum reference frame
Call DatumsStr(annotation)

```

End Sub

```
Private Sub CircularRunoutData(annotation As DimXpertTolerance)
```

```

strs.Add ("")

```

```
strs.Add ("Circular Runout Tolerance Compartment")
```

```
' the tolerance value
```

```
strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
```

```
' the datum reference frame
```

```
Call DatumsStr(annotation)
```

```
End Sub
```

```
Private Sub LineProfileData(annotation As DimXpertLineProfileTolerance)
```

```
Dim I As Double, J As Double, K As Double
```

```
Dim boolstatus As Boolean
```

```
strs.Add ("")
```

```
strs.Add ("Line Profile Tolerance Compartment")
```

```
' the tolerance value
```

```
strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
```

```
' the outer (outside material) tolerance value
```

```
strs.Add (" Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))
```

```
' the vector normal to the profile zones
```

```
strs.Add ("")
```

```
boolstatus = annotation.GetPlanarZoneVector(I, J, K)
```

```
strs.Add ("Planar Zone Vector: " + FormatVector(I, J, K))
```

```
' the datum reference frame
```

```
Call DatumsStr(annotation)
```

```
End Sub
```

```
Private Sub CompositeLineProfileData(annotation As DimXpertCompositeLineProfileTolerance)
```

```
strs.Add ("")
```

```
strs.Add ("Composite Line Profile Tolerance Compartment")
```

```
' the tolerance value
```

```
strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
```

```
' the outer (outside material) tolerance value
```

```
strs.Add (" Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))
```

```
' the vector normal to the profile zones
```

```
strs.Add ("")
```

```
boolstatus = annotation.GetPlanarZoneVector(I, J, K)
```

```

    strs.Add ("Planar Zone Vector: " + FormatVector(I, J, K))

' the datum reference frame
Call DatumsStr(annotation)

' the datum reference frame for the orientation and form
strs.Add ("Composite Datums:")
strs.Add (" Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
strs.Add (" Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True", "False"))
strs.Add (" Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True", "False"))

End Sub

Private Sub SurfaceProfileData(annotation As DimXpertSurfaceProfileTolerance)

    strs.Add ("")
    strs.Add ("Surface Profile Tolerance Compartment")

' the tolerance value
strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

' the outer (outside material) tolerance value
strs.Add (" Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))

' the datum reference frame
Call DatumsStr(annotation)

End Sub

Private Sub CompositeSurfaceProfileData(annotation As
DimXpertCompositeSurfaceProfileTolerance)

    strs.Add ("")
    strs.Add ("Composite Surface Profile Tolerance Compartment")

' the tolerance value
strs.Add (" Tolerance Upper Tier: " + FormatDouble(annotation.Tolerance))

' the outer tolerance value
strs.Add (" Outer Tolerance Upper Tier: " +
FormatDouble(annotation.OuterToleranceValue))

' the datum reference frame for the location
Call DatumsStr(annotation)

' the datum reference frame for the orientation and form
strs.Add ("Composite Datums:")

```

```

    str.Add (" Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
    str.Add (" Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True", "False"))
    str.Add (" Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True", "False"))
End Sub

Private Sub OrientationData(annotation As DimXpertOrientationTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean, value As Double
    Dim annoType As Long
    Dim boolstatus As Boolean
    annoType = annotation.Type
    ' the type or orientation tolerance
    If annoType = swDimXpertGeoTol_Perpendicularity Then
        str.Add ("Orientation Type: Perpendicularity")
    ElseIf annoType = swDimXpertGeoTol_Parallelism Then
        str.Add ("Orientation Type: Parallelism")
    ElseIf annoType = swDimXpertGeoTol_Angularity Then
        str.Add ("Orientation Type: Angularity")
    End If

    str.Add ("")
    str.Add ("Orientation Tolerance Compartment:")

    ' the shape of the tolerance zone
    Select Case annotation.zoneType
    Case swDimXpertOrientationZoneType_Cylindrical
        str.Add (" Zone Type: Cylindrical")
    Case swDimXpertOrientationZoneType_Planar
        str.Add (" Zone Type: Planar")
        boolstatus = annotation.GetPlanarZoneVector(I, J, K)
        str.Add ("Planar Zone Vector: " + FormatVector(I, J, K))
    End Select

    ' the tolerance value
    str.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

    'material condition modifier applied to feature
    str.Add (" Modifier: " + mcmStr(annotation.modifier))

    ' the projected tolerance zone when specified
    boolstatus = annotation.GetProjectedZone(enabled, value)
    Call FormatProjectedZone(enabled, value)

    ' is tangent plane
    str.Add (" IsTangentPlane: " + IIf(annotation.IsTangentPlane, "True", "False"))

```

' the datum reference frame

Call DatumsStr(annotation)

End Sub

Private Sub FlatnessData(annotation As DimXpertFlatnessTolerance)

Dim enabled As Boolean

Dim length As Double, width As Double

Dim I As Double, J As Double, K As Double

Dim boolstatus As Boolean

strs.Add ("")

strs.Add ("Flatness Tolerance Compartment:")

' the tolerance value

strs.Add (" Tolerance: " + FormatDouble(annotation.**Tolerance**))

' the per unit area data

boolstatus = annotation.**GetPerUnitArea**(enabled, length, width, I, J, K)

strs.Add (" Per Unit Area: " + IIf(enabled, "True", "False"))

If enabled Then

strs.Add (" Per Unit Length: " + FormatDouble(length))

strs.Add (" Per Unit Width: " + FormatDouble(width))

strs.Add (" Per Unit Direction: " + FormatVector(I, J, K))

End If

End Sub

Private Sub CircularityData(annotation As DimXpertTolerance)

strs.Add ("")

strs.Add ("Circularity Tolerance Compartment")

' the tolerance value

strs.Add (" Tolerance: " + FormatDouble(annotation.**Tolerance**))

End Sub

Private Sub CylindricityData(annotation As DimXpertTolerance)

strs.Add ("")

strs.Add ("Cylindricity Tolerance Compartment")

' the tolerance value

strs.Add (" Tolerance: " + FormatDouble(annotation.**Tolerance**))

End Sub

Private Sub StraightnessData(annotation As DimXpertStraightnessTolerance)

Dim I As Double, J As Double, K As Double

Dim enabled As Boolean

```

Dim dist As Double
Dim boolstatus As Boolean
'tolerance compartment info
strs.Add ("")
strs.Add ("Straightness Tolerance Compartment")

'type or shape of the tolerance zone
Select Case annotation.zoneType
Case swDimXpertStraightnessZoneType_Cylindrical
    strs.Add (" Zone Type: Cylindrical")
Case swDimXpertStraightnessZoneType_PlanarMedian
    strs.Add (" Zone Type: Planar Median")
Case swDimXpertStraightnessZoneType_Surface
    strs.Add (" Zone Type: Surface Straightness")
    boolstatus = annotation.GetPlanarZoneVector(I, J, K)
    strs.Add (" Zone Vector: " + FormatVector(I, J, K))
End Select

' the tolerance value
strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

'per unit length
boolstatus = annotation.GetPerUnitLength(enabled, dist)
strs.Add (" Per Unit Length: " + IIf(enabled, "True", "False"))
If enabled Then
    strs.Add (" Per Unit Length Distance: " + FormatDouble(dist))
End If

' the material condition modifier
strs.Add (" Modifier: " + mcmStr(annotation.modifier))
End Sub

Private Sub ProfileData(annotation As DimXpertTolerance)
    strs.Add ("")
    strs.Add ("Profile Tolerance Compartment")
    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub SurfaceFinishData(annotation As DimXpertTolerance)
    strs.Add ("")
    strs.Add ("Surface Finish Tolerance Compartment")
    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub

```

```

Private Sub TangencyData(annotation As DimXpertTolerance)
    strs.Add ("")
    strs.Add ("Tangency Tolerance Compartment")
    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub

Private Sub DimensionToleranceData(annotation As DimXpertDimensionTolerance)
    Dim isAngleType As Boolean
    Dim annoType As Long
    Dim upper As Double, lower As Double
    Dim plus As Double, minus As Double
    Dim boolstatus As Boolean
    annoType = annotation.Type

    isAngleType = False

    strs.Add ("")
    strs.Add ("Dimension Tolerance Compartment")
    If annoType = swDimXpertDimTol_DistanceBetween Then
        Dim distancebetween As DimXpertDistanceBetweenDimTol
        Set distancebetween = annotation
        Call DistanceBetweenData(distancebetween)
    ElseIf annoType = swDimXpertDimTol_CompositeDistanceBetween Then
        Dim compdistancebetween As DimXpertCompositeDistanceBetweenDimTol
        Set compdistancebetween = annotation
        Call CompositeDistanceBetweenData(compdistancebetween)

    ElseIf annoType = swDimXpertDimTol_CounterBore Then
        Dim counterBore As IDimXpertCounterBoreDimTol
        Set counterBore = annotation
        If counterBore.ReferenceFeature Is Nothing Then
            strs.Add ("Reference Feature: NULL")
        Else
            strs.Add ("Reference Feature: " + counterBore.ReferenceFeature.Name)
        End If

    ElseIf annoType = swDimXpertDimTol_Depth Then
        Dim depth As IDimXpertDepthDimTol
        Set depth = annotation
        If depth.ReferenceFeature Is Nothing Then
            strs.Add ("Reference Feature: NULL")
        Else
            strs.Add ("Reference Feature: " + depth.ReferenceFeature.Name)
        End If
    End If

```

```

ElseIf annoType = swDimXpertDimTol_CounterSinkDiameter Then
    Dim countersink As IDimXpertCounterSinkDiameterDimTol
    Set countersink = annotation
    If countersink.ReferenceFeature Is Nothing Then
        str$.Add ("Reference Feature: NULL")
    Else
        str$.Add ("Reference Feature: " + countersink.ReferenceFeature.Name)
    End If

ElseIf annoType = swDimXpertDimTol_ChamferDimension Then

    Select Case annotation.ChamferType
    Case swDimXpertChamferDimensionType_Angle
        str$.Add ("Chamfer Dimension Type: Angle")
        isAngleType = True
    Case swDimXpertChamferDimensionType_LinearDistance1
        str$.Add ("Chamfer Dimension Type: Distance 1")
    Case swDimXpertChamferDimensionType_LinearDistance2
        str$.Add ("Chamfer Dimension Type: Distance 2")
    End Select

ElseIf annoType = swDimXpertDimTol_AngleBetween Then
    isAngleType = True
    Dim angleBetween As IDimXpertAngleBetweenDimTol
    Set angleBetween = annotation
    ' the origin and tolerance feature
    str$.Add ("Origin Feature: " + angleBetween.OriginFeature.Name)

    ' is supplement angle
    str$.Add ("Supplement Angle: " + IIf(angleBetween.Supplement, "True", "False"))

ElseIf annoType = swDimXpertDimTol_CounterSinkAngle Then
    isAngleType = True
    Dim countersinkAngle As IDimXpertCounterSinkAngleDimTol
    Set countersinkAngle = annotation

    If countersinkAngle.ReferenceFeature Is Nothing Then
        str$.Add ("Reference Feature: NULL")
    Else
        str$.Add ("Reference Feature: " + countersinkAngle.ReferenceFeature.Name)
    End If

```

```

ElseIf annoType = swDimXpertDimTol_ConeAngle Then
    isAngleType = True
End If

' conversion for radians to degrees when dimension type is angle
Dim dbl As Double
If isAngleType Then
    dbl = 57.2957795130823
Else
    dbl = 1#
End If

' the nominal, and upper and lower limits of size of the dimension
strs.Add ("")
strs.Add ("Compute Nominal Dimension: " + FormatDouble(annotation.GetNominalValue * dbl))
boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)
strs.Add ("Get Upper Limit: " + FormatDouble(upper * dbl))
strs.Add ("Get Lower Limit: " + FormatDouble(lower * dbl))

' the upper and lower tolerance value by type
Select Case annotation.DimensionType

Case swDimXpertDimTolType_BlockTolerance
    strs.Add ("Dimension Type: Block Tolerance")

' block tolerance
Case swDimXpertDimTolType_BlockToleranceNoNominal
    Dim blockTols As DimXpertBlockTolerances
    Set blockTols = dimXpertPart.GetBlockTolerances

    Select Case blockTols.Type
        Case swDimXpertBlockToleranceType_ASMEInch
            strs.Add ("Dimension Type: Block Tolerance No Nominal")
            If isAngleType Then
                strs.Add ("Angular Block Tolerance")
            Else
                strs.Add ("Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlaces, "##,##0"))
            End If
        Case swDimXpertBlockToleranceType_ISO2768
            strs.Add ("Dimension Type: General Tolerance")
    End Select

Case swDimXpertDimTolType_ISOLimitsAndFits
    strs.Add ("Dimension Type: Limits and Fits")
' limits and fits tolerance

```

```

Case swDimXpertDimTolType_ISOLimitsAndFitsNoNominal
    str.Add ("Dimension Type: Limits and Fits No Nominal")
    str.Add ("Limits and Fits: " + annotation.LimitsAndFitsCode)
' limit dimension
Case swDimXpertDimTolType_LimitDimension
    str.Add ("Dimension Type: Limit Dimension")
    boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)
    str.Add ("Get Upper Limit: " + FormatDouble(upper * dbl))
    str.Add ("Get Lower Limit: " + FormatDouble(lower * dbl))
Case swDimXpertDimTolType_MAXTolerance
    str.Add ("Dimension Type: MAXTolerance")
Case swDimXpertDimTolType_MINTolerance
    str.Add ("Dimension Type: MINTolerance")
Case swDimXpertDimTolType_NoTolerance
    str.Add ("Dimension Type: NoTolerance")
Case swDimXpertDimTolType_PlusMinusDimension
    str.Add ("Dimension Type: Plus Minus Dimension")

```

```

' plus and minus dimension

```

```

Case swDimXpertDimTolType_PlusMinusNoNominal
    str.Add ("Dimension Type: Plus Minus No Nominal")
    boolstatus = annotation.GetPlusAndMinusTolerance(plus, minus)
    str.Add ("Plus Tolerance: " + FormatDouble(plus * dbl))
    str.Add ("Minus Tolerance: " + FormatDouble(minus * dbl))

```

```

End Select

```

```

End Sub

```

```

Private Sub DistanceBetweenData(annotation As DimXpertDistanceBetweenDimTol)
    Dim feature As DimXpertFeature
    Dim featureFosUsage As Long
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    ' the origin and tolerance feature along with their feature of size usage (min, max, center)
    boolstatus = annotation.GetOriginFeature(feature, featureFosUsage)
    str.Add ("")
    str.Add ("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
    boolstatus = annotation.GetFeature(feature, featureFosUsage)
    str.Add ("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))

    ' The direction vector
    boolstatus = annotation.GetDirectionVector(I, J, K)
    str.Add ("")
    str.Add ("Direction Vector: " + FormatVector(I, J, K))

```

End Sub

Private Sub CompositeDistanceBetweenData(annotation As
DimXpertCompositeDistanceBetweenDimTol)

Dim feature As DimXpertFeature

Dim featureFosUsage As Long

Dim plus As Double, minus As Double

Dim I As Double, J As Double, K As Double

Dim boolstatus As Boolean

Dim blockTols As DimXpertBlockTolerances

' the origin and tolerance feature along with their feature of size usage (min, max,
center)

boolstatus = annotation.**GetOriginFeature**(feature, featureFosUsage)

strs.Add ("")

strs.Add ("Origin Feature: " + feature.**Name** + " @ " + FosUsage(featureFosUsage))

boolstatus = annotation.**GetFeature**(feature, featureFosUsage)

strs.Add ("Tolerance Feature: " + feature.**Name** + " @ " + FosUsage(featureFosUsage))

' the pattern locating feature

boolstatus = annotation.**GetIntraFeature**(feature, featureFosUsage)

strs.Add ("Pattern Locating Feature: " + feature.**Name** + " @ " +
FosUsage(featureFosUsage))

' The direction vector

boolstatus = annotation.**GetDirectionVector**(I, J, K)

strs.Add ("")

strs.Add ("Direction Vector: " + FormatVector(I, J, K))

' the upper and lower tolerance value for the pattern location by type

Select Case annotation.**DimensionType**

' plus and minus dimension

Case swDimXpertDimTolType_PlusMinusNoNominal

strs.Add ("Pattern Locating Dimension Type: Plus Minus No Nominal")

boolstatus = annotation.**GetPlusAndMinusToleranceIntraFeature**(plus, minus)

strs.Add (" Plus Tolerance: " + FormatDouble(plus))

strs.Add (" Minus Tolerance: " + FormatDouble(minus))

' block tolerance

Case swDimXpertDimTolType_BlockToleranceNoNominal

Set blockTols = dimXpertPart.**GetBlockTolerances**

Select Case blockTols.**Type**

Case swDimXpertBlockToleranceType_ASMEInch

```

        str.Add ("Pattern Locating Dimension Type: Block Tolerance No Nominal")
        str.Add ("  Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlaces, "##,##0"))
        Case swDimXpertBlockToleranceType_IS02768
            str.Add ("Pattern Locating Dimension Type: General Tolerance")
        End Select

End Select

' the upper and lower tolerance value for the feature to feature location by type
Select Case annotation.DimensionTypeIntraFeature
' plus and minus dimension
Case swDimXpertDimTolType_PlusMinusNoNominal
    str.Add ("Feature Locating Dimension Type: Plus Minus No Nominal")
    boolstatus = annotation.GetPlusAndMinusToleranceIntraFeature(plus, minus)
    str.Add ("  Plus  Tolerance: " + FormatDouble(plus))
    str.Add ("  Minus Tolerance: " + FormatDouble(minus))
' block tolerance
Case swDimXpertDimTolType_BlockToleranceNoNominal

    Set blockTols = dimXpertPart.GetBlockTolerances
    Select Case blockTols.Type
        Case swDimXpertBlockToleranceType_ASMEInch
            str.Add ("Feature locating Dimension Type: Block Tolerance No Nominal")
            str.Add ("  Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlacesIntraFeature, "##,##0"))
            Case swDimXpertBlockToleranceType_IS02768
                str.Add ("Feature locating Dimension Type: General Tolerance")
            End Select
        End Select
    End Select

End Sub

Private Sub DatumsStr(annotation As DimXpertTolerance)
    str.Add ("")
    str.Add ("Datums:")
    Call datumStr(annotation.GetPrimaryDatums(), annotation.GetPrimaryDatumModifiers(), "
Primary:")
    Call datumStr(annotation.GetSecondaryDatums(), annotation.GetSecondaryDatumModifiers(), "
Secondary:")
    Call datumStr(annotation.GetTertiaryDatums(), annotation.GetTertiaryDatumModifiers(), "
Tertiary:")

End Sub

Private Sub datumStr(dats As Variant, mods As Variant, datumOrder As String)
    Dim I    As Long
    Dim str As String

```

```

Dim mcm As Long
If IsEmpty(dats) Then
    strs.Add (datumOrder + ": none")
    Exit Sub
End If

For I = LBound(dats) To UBound(dats)
    mcm = mods(I)
    str = str + " " + dats(I).Identifier + " @ " + mcmStr(mcm)
Next I

If StrComp(str, "") > 0 Then
    strs.Add (datumOrder + str)
Else
    strs.Add (datumOrder + " <none>")
End If
End Sub

' returns a string containing the height of the projected tolerance zone
Private Sub FormatProjectedZone(enabled As Boolean, height As Double)

    If enabled = True Then
        strs.Add (" Projected Zone: True")
        strs.Add (" Zone Height: " + FormatDouble(height))
    Else
        strs.Add (" Projected Zone: False")
    End If

End Sub

End Sub
Private Function mcmStr(mcm As Long) As String
    Dim str

    Select Case mcm
    Case swDimXpertMCM_LMC
        str = "LMC"
    Case swDimXpertMCM_MMC
        str = "MMC"
    Case swDimXpertMCM_NoMCM
        str = "NoMCM"
    Case swDimXpertMCM_RFS
        str = "RFS"
    End Select

    mcmStr = str
End Function

' returns a string containing the type of position tolerance zone used

```

```

Private Function PositionZoneType(typ As Long) As String
    Dim str As String

    Select Case typ
    Case swDimXpertPositionZoneType_CylindricalPosition
        str = "Cylindrical"
    Case swDimXpertPositionZoneType_PlanarPosition
        str = "Planar"
    Case swDimXpertPositionZoneType_SphericalPosition
        str = "Spherical"
    Case swDimXpertPositionZoneType_Boundary
        str = "Boundary"
    Case swDimXpertPositionZoneType_RadialPositionArc
        str = "RadialPositionArc"
    Case swDimXpertPositionZoneType_RadialPositionPlanar
        str = "RadialPositionPlanar"
    Case Else
        str = "N/A"
    End Select

    PositionZoneType = str
End Function

' returns a string containing the names of the SW display entities
Private Function DisplayEntity(annotation As DimXpertAnnotation) As String
    Dim str As String
    Dim dispEnt As Object
    Dim swAnnot As SldWorks.annotation

    'Set dispEnt = swDimXpert.GetDisplayEntity(annot)
    Set dispEnt = annotation.GetDisplayEntity

    If Not dispEnt Is Nothing Then
        If TypeOf dispEnt Is SldWorks.annotation Then
            Set swAnnot = dispEnt
            str = swAnnot.GetName
        End If
    End If

    DisplayEntity = str
End Function

' returns a string containing the feature of size usage (min, max, center)
Private Function FosUsage(value As Long) As String
    Dim str
    Select Case value

```

```

    Case swDimXpertDistanceFosUsage_Center
        str = "Center"
    Case swDimXpertDistanceFosUsage_MaximumSide
        str = "Max"
    Case swDimXpertDistanceFosUsage_MinimumSide
        str = "Min"
    Case Else
        str = "N/A"
    End Select
    FosUsage = str
End Function

Private Function FormatVector(I As Double, J As Double, K As Double) As String
    Dim str
    str = FormatDouble(I) + ", " + FormatDouble(J) + ", " + FormatDouble(K)

    FormatVector = str
End Function

Private Function FormatDouble(value As Double) As String
    Dim str

    str = Format(value, "##,##0.0000")

    FormatDouble = str
End Function

Private Function RadiansToDegrees(value As Double) As String
    Dim str

    str = Format((value * 57.2957795130823), "##,##0.00")

    RadiansToDegrees = str
End Function

Private Function annotationTypeNameFromObject(anno As DimXpertAnnotation) As String
    annotationTypeNameFromObject = annotationTypeNameFromTypeNumber(anno.Type)
End Function

Private Function annotationTypeNameFromTypeNumber(annoTypeIndex As Long) As String
    Select Case annoTypeIndex

        Case swDimXpertDimTol_DistanceBetween
            annotationTypeNameFromTypeNumber = "DistanceBetween Dim"
        Case swDimXpertDimTol_CounterBore
            annotationTypeNameFromTypeNumber = "CounterBore Dim"
        Case swDimXpertDimTol_Depth
            annotationTypeNameFromTypeNumber = "Depth Dim"
        Case swDimXpertDimTol_CounterSinkDiameter
            annotationTypeNameFromTypeNumber = "CounterSinkDiameter Dim"
    End Select
End Function

```

```
Case swDimXpertDimTol_ChamferDimension
    annotationTypeNameFromTypeNumber = "ChamferDimension Dim"
Case swDimXpertDimTol_AngleBetween
    annotationTypeNameFromTypeNumber = "AngleBetween Dim"
Case swDimXpertDimTol_CounterSinkAngle
    annotationTypeNameFromTypeNumber = "CounterSinkAngle Dim"
Case swDimXpertDimTol_ConeAngle
    annotationTypeNameFromTypeNumber = "ConeAngle Dim"
Case swDimXpertDimTol_Diameter
    annotationTypeNameFromTypeNumber = "Diameter Dim"
Case swDimXpertDimTol_Length
    annotationTypeNameFromTypeNumber = "Length Dim"
Case swDimXpertDimTol_Radius
    annotationTypeNameFromTypeNumber = "Radius Dim"
Case swDimXpertDimTol_Width
    annotationTypeNameFromTypeNumber = "Width Dim"
Case swDimXpertDimTol_CompositeDistanceBetween
    annotationTypeNameFromTypeNumber = "CompositeDistanceBetween Dim"
Case swDimXpertDatum
    annotationTypeNameFromTypeNumber = "Datum"
Case swDimXpertGeoTol_Position
    annotationTypeNameFromTypeNumber = "Position Tol"
Case swDimXpertGeoTol_CompositePosition
    annotationTypeNameFromTypeNumber = "CompositePosition Tol"
Case swDimXpertGeoTol_Symmetry
    annotationTypeNameFromTypeNumber = "Symmetry Tol"
Case swDimXpertGeoTol_Concentricity
    annotationTypeNameFromTypeNumber = "Concentricity Tol"
Case swDimXpertGeoTol_LineProfile
    annotationTypeNameFromTypeNumber = "LineProfile Tol"
Case swDimXpertGeoTol_CompositeLineProfile
    annotationTypeNameFromTypeNumber = "CompositeLineProfile Tol"
Case swDimXpertGeoTol_SurfaceProfile
    annotationTypeNameFromTypeNumber = "SurfaceProfile Tol"
Case swDimXpertGeoTol_CompositeSurfaceProfile
    annotationTypeNameFromTypeNumber = "CompositeSurfaceProfile Tol"
Case swDimXpertGeoTol_Angularity
    annotationTypeNameFromTypeNumber = "Angularity Tol"
Case swDimXpertGeoTol_Parallelism
    annotationTypeNameFromTypeNumber = "Parallelism Tol"
Case swDimXpertGeoTol_Perpendicularity
    annotationTypeNameFromTypeNumber = "Perpendicularity Tol"
Case swDimXpertGeoTol_TotalRunout
    annotationTypeNameFromTypeNumber = "TotalRunout Tol"
Case swDimXpertGeoTol_CircularRunout
```

```

        annotationTypeNameFromTypeNumber = "CircularRunout Tol"
Case swDimXpertGeoTol_Flatness
        annotationTypeNameFromTypeNumber = "Flatness Tol"
Case swDimXpertGeoTol_Circularity
        annotationTypeNameFromTypeNumber = "Circularity Tol"
Case swDimXpertGeoTol_Cylindricity
        annotationTypeNameFromTypeNumber = "Cylindricity Tol"
Case swDimXpertGeoTol_Straightness
        annotationTypeNameFromTypeNumber = "Straightness Tol"
Case swDimXpertGeoTol_Tangency
        annotationTypeNameFromTypeNumber = "Tangency Tol"
Case Else
        annotationTypeNameFromTypeNumber = "<unknown> " & CStr(annoTypeIndex)

End Select
End Function

```

Get DimXpert Tolerance Example (VB.NET)

This example shows how to build a part and get attributes for the following DimXpert annotations:

- * Counterbore dimension tolerance
- * Depth dimension tolerance

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\face_plate_ads_geo.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Click Auto Dimension Scheme on the DimXpert toolbar.  
' 4. Ensure that all feature filters are selected.  
' 5. Click the green check mark to accept the settings.  
' 6. In the DimXpertManager tab of the Management Panel,  
'    expand the Hole Pattern8 node in the DimXpertManager tab of the  
'    Management Panel.  
' 7. Observe the following Dimxpert annotations: Counterbore3 and Depth3  
' 8. Open an Immediate window.  
' 9. Ensure that the latest SolidWorks.Interop.swdimxpert.dll interop assembly  
'    is loaded (right-click the project in Project Explorer and  
'    click Add Reference > .NET tab).  
' 10. Ensure that the Microsoft Scripting Runtime library is loaded  
'    (right-click the project in Project Explorer and click Add Reference >  
'    COM tab).  
'  
' Postconditions:  
' 1. Writes the output to c:\temp\dimXpertInfo.txt.  
' 2. Inspect the Immediate Window.  
'  
' NOTE: Because these parts are used elsewhere, do not save changes.  
' -----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swdimxpert  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics  
Imports Scripting  
Partial Class SolidWorksMacro  
    Dim str As New Collection  
    Dim boolstatus As Boolean  
    Dim dimXpertPart As DimXpertPart  
    Dim i As Double
```

```

Dim j As Double
Dim k As Double
Sub Main()
    Dim swModelDoc As IModelDoc2
    swModelDoc = swApp.ActiveDoc
    If swModelDoc Is Nothing Then
        Exit Sub
    End If
    Dim fso As New Scripting.FileSystemObject()
    Dim f As Scripting.File
    Dim textStr As Scripting.TextStream
    textStr = fso.CreateTextFile("c:\temp\dimXpertInfo.txt")
    f = fso.GetFile("c:\temp\dimXpertInfo.txt")
    'textStr = f.OpenAsTextStream(2, -2)
    If textStr Is Nothing Then
        Debug.Print("Error creating temp file.")
        Exit Sub
    End If
    Call log("Starting DimXpert log...", textStr)
    Call retrieve_info_text(swApp, textStr)
    textStr.Close()
End Sub

Private Sub log(ByVal text As String, ByVal textStr As Scripting.TextStream)
    Debug.Print(text)
    textStr.WriteLine(text)
End Sub

Private Sub retrieve_info_text(ByVal swapp As SldWorks, ByVal textStr As TextStream)
    Dim dimXpertMgr As SolidWorks.Interop.sldworks.DimXpertManager
    dimXpertMgr =
swapp.IActiveDoc2.Extension.DimXpertManager(swapp.IActiveDoc2.IGetActiveConfiguration().Name,
True)

    Call log("Model: " & swapp.IActiveDoc2.GetPathName, textStr)
    Dim dimXpertPartObj As DimXpertPart
    dimXpertPartObj = dimXpertMgr.DimXpertPart
    dimXpertPart = dimXpertPartObj
    Dim vAnnotations As Object
    vAnnotations = dimXpertPart.GetAnnotations()
    Call log("-----", textStr)
    Call log("Annotations...", textStr)
    Call log("-----", textStr)
    Dim annotationTemp As DimXpertAnnotation
    Dim annotationIndex As Long
    For annotationIndex = 0 To UBound(vAnnotations)
        annotationTemp = vAnnotations(annotationIndex)
        Dim AnnotationDataText As Collection

```

```

        AnnotationDataText = AnnotationData(annotationTemp)
        Dim annotationTextIndex As Long
        For annotationTextIndex = 1 To AnnotationDataText.Count
            Call log(AnnotationDataText(annotationTextIndex), textStr)
        Next
    Next
End Sub

Public Function AnnotationData(ByVal annotation As DimXpertAnnotation) As Collection
    Dim annoType As Long
    'general info
    Call GeneralInfo(annotation)
    annoType = annotation.Type
    If annoType = swDimXpertAnnotationType_e.swDimXpertDatum Then
        Call DatumData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Position Then
        Call PositionData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositePosition Then
        Call CompositePositionData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Symmetry Then
        Call SymmetryData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Concentricity Then
        Call ConcentricityData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_LineProfile Then
        Call LineProfileData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeLineProfile
Then
        Call CompositeLineProfileData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_SurfaceProfile Then
        Call SurfaceProfileData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeSurfaceProfile
Then
        Call CompositeSurfaceProfileData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Angularity Or annoType
= swDimXpertAnnotationType_e.swDimXpertGeoTol_Parallelism Or annoType =
swDimXpertAnnotationType_e.swDimXpertGeoTol_Perpendicularity Then
        Call OrientationData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_TotalRunout Then
        Call TotalRunoutData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_CircularRunout Then
        Call CircularRunoutData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Flatness Then
        Call FlatnessData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Circularity Then
        Call CircularityData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Cylindricity Then
        Call CylindricityData(annotation)

```

```

    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Straightness Then
        Call StraightnessData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Tangency Then
        Call TangencyData(annotation)
    Else ' any of the dimension tolerance types
        Call DimensionToleranceData(annotation)
    End If
    AnnotationData = strs
End Function
Private Sub Clear(ByVal strs As Collection)
    strs.Remove(strs.Count)
    If Not strs.Count = 0 Then
        Call Clear(strs)
    End If
End Sub
Private Sub GeneralInfo(ByVal annotation As DimXpertAnnotation)
    Dim annoType As String
    Dim modelObj As Feature
    If Not strs.Count = 0 Then
        Call Clear(strs)
    End If
    strs.Add("")
    strs.Add("Name: " + annotation.Name)
    annoType = annotationTypeNameFromObject(annotation)
    strs.Add("Type: " + annoType)
    strs.Add("Display Entity: " + DisplayEntity(annotation))
    modelObj = annotation.GetModelFeature()
    If Not (modelObj Is Nothing) Then
        strs.Add("ModelFeature: " + modelObj.Name + " (" + modelObj.GetTypeName2() + ")")
    End If
End Sub
Private Sub DatumData(ByVal annotation As DimXpertDatum)
    ' the datum letter
    strs.Add("")
    strs.Add("Datum Letter: " + annotation.Identifier)
End Sub
Private Sub PositionData(ByVal annotation As DimXpertPositionTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean, value As Double
    Dim boolstatus As Boolean
    strs.Add("")
    strs.Add("Position Tolerance Compartment:")
    ' the shape of the tolerance zone
    strs.Add(" Zone Type: " + PositionZoneType(annotation.ZoneType))
    ' the zone vector if the tolerance zone is planar

```

```

        If annotation.ZoneType =
swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_PlanarPosition Then
            boolstatus = annotation.GetPlanarZoneVector(I, J, K)
            str$.Add(" Direction: " + FormatVector(I, J, K))
        End If
        ' the tolerance value
        str$.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
        ' the material condition modifier applied to feature
        str$.Add(" Modifier: " + mcmStr(annotation.Modifier))
        ' the projected tolerance zone when specified
        boolstatus = annotation.GetProjectedZone(enabled, value)
        Call FormatProjectedZone(enabled, value)
        ' the datum reference frame
        Call DatumsStr(annotation)
    End Sub

Private Sub CompositePositionData(ByVal annotation As DimXpertCompositePositionTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean, value As Double
    Dim boolstatus As Boolean
    str$.Add("")
    str$.Add("Composite Position Tolerance Compartment")
    ' the shape of the tolerance zone
    str$.Add(" Zone Type: " + PositionZoneType(annotation.ZoneType))
    ' the zone vector when the zone is planar
    If annotation.ZoneType =
swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_PlanarPosition Then
        boolstatus = annotation.GetPlanarZoneVector(I, J, K)
        str$.Add(" Direction: " + FormatVector(I, J, K))
    End If
    ' the tolerance value
    str$.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the material condition modifier
    str$.Add(" Modifier: " + mcmStr(annotation.Modifier))
    ' the projected tolerance zone when specified
    boolstatus = annotation.GetProjectedZone(enabled, value)
    Call FormatProjectedZone(enabled, value)
    ' the datum reference frame for the pattern location
    Call DatumsStr(annotation)
    ' the datum reference frame for the feature to feature location
    str$.Add("Composite datums:")
    str$.Add(" Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
    str$.Add(" Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True",
"False"))
    str$.Add(" Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True",
"False"))
End Sub

```

```

Private Sub SymmetryData(ByVal annotation As DimXpertSymmetryTolerance)
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    strs.Add("")
    strs.Add("Symmetry Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the material condition modifier applied to feature
    strs.Add("  Modifier: " + mcmStr(annotation.Modifier))
    ' the datum reference frame
    strs.Add("")
    Call DatumsStr(annotation)
    ' the direction of the planar zone
    strs.Add("")
    boolstatus = annotation.GetZoneDirection(I, J, K)
    strs.Add("Planar Zone Direction: " + FormatVector(I, J, K))
End Sub

Private Sub ConcentricityData(ByVal annotation As DimXpertConcentricityTolerance)
    strs.Add("")
    strs.Add("Concentricity Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the material condition modifier applied to feature
    strs.Add("  Modifier: " + mcmStr(annotation.Modifier))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub TotalRunoutData(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Total Runout Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub CircularRunoutData(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Circular Runout Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub LineProfileData(ByVal annotation As DimXpertLineProfileTolerance)
    Dim I As Double, J As Double, K As Double

```

```

    Dim boolstatus As Boolean
    strs.Add("")
    strs.Add("Line Profile Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the outer (outside material) tolerance value
    strs.Add("  Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))
    ' the vector normal to the profile zones
    strs.Add("")
    boolstatus = annotation.GetPlanarZoneVector(I, J, K)
    strs.Add("Planar Zone Vector: " + FormatVector(I, J, K))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub CompositeLineProfileData(ByVal annotation As
DimXpertCompositeLineProfileTolerance)
    strs.Add("")
    strs.Add("Composite Line Profile Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the outer (outside material) tolerance value
    strs.Add("  Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))
    ' the vector normal to the profile zones
    strs.Add("")
    boolstatus = annotation.GetPlanarZoneVector(i, j, k)
    strs.Add("Planar Zone Vector: " + FormatVector(i, j, k))
    ' the datum reference frame
    Call DatumsStr(annotation)
    ' the datum reference frame for the orientation and form
    strs.Add("Composite Datums:")
    strs.Add("  Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
    strs.Add("  Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True",
"False"))
    strs.Add("  Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True",
"False"))
End Sub

Private Sub SurfaceProfileData(ByVal annotation As DimXpertSurfaceProfileTolerance)
    strs.Add("")
    strs.Add("Surface Profile Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the outer (outside material) tolerance value
    strs.Add("  Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

```

```

Private Sub CompositeSurfaceProfileData(ByVal annotation As
DimXpertCompositeSurfaceProfileTolerance)
    strs.Add("")
    strs.Add("Composite Surface Profile Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance Upper Tier: " + FormatDouble(annotation.Tolerance))
    ' the outer tolerance value
    strs.Add("  Outer Tolerance Upper Tier: " +
FormatDouble(annotation.OuterToleranceValue))
    ' the datum reference frame for the location
    Call DatumsStr(annotation)
    ' the datum reference frame for the orientation and form
    strs.Add("Composite Datums:")
    strs.Add("  Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
    strs.Add("  Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True",
"False"))
    strs.Add("  Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True",
"False"))
End Sub

Private Sub OrientationData(ByVal annotation As DimXpertOrientationTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean, value As Double
    Dim annoType As Long
    annoType = annotation.Type
    ' the type or orientation tolerance
    If annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Perpendicularity Then
        strs.Add("Orientation Type: Perpendicularity")
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Parallelism Then
        strs.Add("Orientation Type: Parallelism")
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Angularity Then
        strs.Add("Orientation Type: Angularity")
    End If
    strs.Add("")
    strs.Add("Orientation Tolerance Compartment:")
    ' the shape of the tolerance zone
    Select Case annotation.ZoneType
        Case swDimXpertOrientationZoneType_e.swDimXpertOrientationZoneType_Cylindrical
            strs.Add("  Zone Type: Cylindrical")
        Case swDimXpertOrientationZoneType_e.swDimXpertOrientationZoneType_Planar
            strs.Add("  Zone Type: Planar")
            boolstatus = annotation.GetPlanarZoneVector(I, J, K)
            strs.Add("Planar Zone Vector: " + FormatVector(I, J, K))
    End Select
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    'material condition modifier applied to feature

```

```

    strs.Add("  Modifier: " + mcmStr(annotation.Modifier))
    ' the projected tolerance zone when specified
    boolstatus = annotation.GetProjectedZone(enabled, value)
    Call FormatProjectedZone(enabled, value)
    ' is tangent plane
    strs.Add("  IsTangentPlane: " + IIf(annotation.IsTangentPlane, "True", "False"))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub FlatnessData(ByVal annotation As DimXpertFlatnessTolerance)
    Dim enabled As Boolean
    Dim length As Double, width As Double
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    strs.Add("")
    strs.Add("Flatness Tolerance Compartment:")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the per unit area data
    boolstatus = annotation.GetPerUnitArea(enabled, length, width, I, J, K)
    strs.Add("  Per Unit Area: " + IIf(enabled, "True", "False"))
    If enabled Then
        strs.Add("    Per Unit Length: " + FormatDouble(length))
        strs.Add("    Per Unit Width: " + FormatDouble(width))
        strs.Add("    Per Unit Direction: " + FormatVector(I, J, K))
    End If
End Sub

Private Sub CircularityData(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Circularity Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub

Private Sub CylindricityData(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Cylindricity Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub

Private Sub StraightnessData(ByVal annotation As DimXpertStraightnessTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean
    Dim dist As Double
    Dim boolstatus As Boolean
    'tolerance compartment info

```

```

    strs.Add("")
    strs.Add("Straightness Tolerance Compartment")
    'type or shape of the tolerance zone
    Select Case annotation.ZoneType
        Case swDimXpertStraightnessZoneType_e.swDimXpertStraightnessZoneType_Cylindrical
            strs.Add(" Zone Type: Cylindrical")
        Case swDimXpertStraightnessZoneType_e.swDimXpertStraightnessZoneType_PlanarMedian
            strs.Add(" Zone Type: Planar Median")
        Case swDimXpertStraightnessZoneType_e.swDimXpertStraightnessZoneType_Surface
            strs.Add(" Zone Type: Surface Straightness")
            boolstatus = annotation.GetPlanarZoneVector(I, J, K)
            strs.Add(" Zone Vector: " + FormatVector(I, J, K))
    End Select
    ' the tolerance value
    strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
    'per unit length
    boolstatus = annotation.GetPerUnitLength(enabled, dist)
    strs.Add(" Per Unit Length: " + IIf(enabled, "True", "False"))
    If enabled Then
        strs.Add(" Per Unit Length Distance: " + FormatDouble(dist))
    End If
    ' the material condition modifier
    strs.Add(" Modifier: " + mcmStr(annotation.Modifier))
End Sub

Private Sub ProfileData(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Profile Tolerance Compartment")
    ' the tolerance value
    strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub SurfaceFinishData(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Surface Finish Tolerance Compartment")
    ' the tolerance value
    strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub

Private Sub TangencyData(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Tangency Tolerance Compartment")
    ' the tolerance value
    strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub

Private Sub DimensionToleranceData(ByVal annotation As DimXpertDimensionTolerance)

```

```

Dim isAngleType As Boolean
Dim annoType As Long
Dim upper As Double, lower As Double
Dim plus As Double, minus As Double
Dim boolstatus As Boolean
annoType = annotation.Type
isAngleType = False
strs.Add("")
strs.Add("Dimension Tolerance Compartment")
If annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_DistanceBetween Then
    Call DistanceBetweenData(annotation)
ElseIf annoType =
swDimXpertAnnotationType_e.swDimXpertDimTol_CompositeDistanceBetween Then
    Call CompositeDistanceBetweenData(annotation)
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_CounterBore Then
    Dim cbAnno As IDimXpertCounterBoreDimTol
    cbAnno = annotation
    If cbAnno.ReferenceFeature Is Nothing Then
        strs.Add("Reference Feature: NULL")
    Else
        strs.Add("Reference Feature: " + cbAnno.ReferenceFeature.Name)
    End If
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_Depth Then
    Dim depthAnno As IDimXpertDepthDimTol
    depthAnno = annotation
    If depthAnno.ReferenceFeature Is Nothing Then
        strs.Add("Reference Feature: NULL")
    Else
        strs.Add("Reference Feature: " + depthAnno.ReferenceFeature.Name)
    End If
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkDiameter
Then
    If annotation.ReferenceFeature Is Nothing Then
        strs.Add("Reference Feature: NULL")
    Else
        strs.Add("Reference Feature: " + annotation.ReferenceFeature.Name)
    End If
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_ChamferDimension Then
    Dim chamferAnno As IDimXpertChamferDimTol
    chamferAnno = annotation
    Select Case chamferAnno.ChamferType
        Case swDimXpertChamferDimensionType_e.swDimXpertChamferDimensionType_Angle
            strs.Add("Chamfer Dimension Type: Angle")
            isAngleType = True
        Case
swDimXpertChamferDimensionType_e.swDimXpertChamferDimensionType_LinearDistance1

```

```

        str.Add("Chamfer Dimension Type: Distance 1")
    Case
swDimXpertChamferDimensionType_e.swDimXpertChamferDimensionType_LinearDistance2
        str.Add("Chamfer Dimension Type: Distance 2")
    End Select
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_AngleBetween Then
    isAngleType = True
    ' the origin and tolerance feature
    str.Add("Origin Feature: " + annotation.OriginFeature.Name)
    ' is supplement angle
    str.Add("Supplement Angle: " + IIf(annotation.Supplement, "True", "False"))
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkAngle Then
    isAngleType = True
    If annotation.ReferenceFeature Is Nothing Then
        str.Add("Reference Feature: NULL")
    Else
        str.Add("Reference Feature: " + annotation.ReferenceFeature.Name)
    End If
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_ConeAngle Then
    isAngleType = True
End If
' conversion for radians to degrees when dimension type is angle
Dim dbl As Double
If isAngleType Then
    dbl = 57.2957795130823
Else
    dbl = 1.0#
End If
' the nominal, and upper and lower limits of size of the dimension
str.Add("")
str.Add("Compute Nominal Dimension: " + FormatDouble(annotation.GetNominalValue *
dbl))
boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)
str.Add("Get Upper Limit: " + FormatDouble(upper * dbl))
str.Add("Get Lower Limit: " + FormatDouble(lower * dbl))
' the upper and lower tolerance value by type
Select Case annotation.DimensionType
    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockTolerance
        str.Add("Dimension Type: Block Tolerance")
        ' block tolerance
    Case
swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockToleranceNoNominal
        Dim blockTols As DimXpertBlockTolerances
        blockTols = dimXpertPart.GetBlockTolerances
        Select Case blockTols.Type
            Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ASMEInch

```

```

        strs.Add("Dimension Type: Block Tolerance No Nominal")
        If isAngleType Then
            strs.Add("Angular Block Tolerance")
        Else
            strs.Add("Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlaces, "##,##0"))
        End If
        Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ISO2768
            strs.Add("Dimension Type: General Tolerance")
        End Select
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_ISOLimitsAndFits
            strs.Add("Dimension Type: Limits and Fits")
            ' limits and fits tolerance
        Case
swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_ISOLimitsAndFitsNoNominal
            strs.Add("Dimension Type: Limits and Fits No Nominal")
            strs.Add("Limits and Fits: " + annotation.LimitsAndFitsCode)
            ' limit dimension
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_LimitDimension
            strs.Add("Dimension Type: Limit Dimension")
            boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)
            strs.Add("Get Upper Limit: " + FormatDouble(upper * dbl))
            strs.Add("Get Lower Limit: " + FormatDouble(lower * dbl))
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_MAXTolerance
            strs.Add("Dimension Type: MAXTolerance")
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_MINTolerance
            strs.Add("Dimension Type: MINTolerance")
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_NoTolerance
            strs.Add("Dimension Type: NoTolerance")
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusDimension
            strs.Add("Dimension Type: Plus Minus Dimension")
            ' plus and minus dimension
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusNoNominal
            strs.Add("Dimension Type: Plus Minus No Nominal")
            boolstatus = annotation.GetPlusAndMinusTolerance(plus, minus)
            strs.Add("Plus Tolerance: " + FormatDouble(plus * dbl))
            strs.Add("Minus Tolerance: " + FormatDouble(minus * dbl))
        End Select
    End Sub

Private Sub DistanceBetweenData(ByVal annotation As DimXpertDistanceBetweenDimTol)
    Dim feature As DimXpertFeature
    Dim featureFosUsage As Long
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    feature = Nothing

```

```

' the origin and tolerance feature along with their feature of size usage (min, max,
center)
boolstatus = annotation.GetOriginFeature(feature, featureFosUsage)
strs.Add("")
strs.Add("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
boolstatus = annotation.GetFeature(feature, featureFosUsage)
strs.Add("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
' The direction vector
boolstatus = annotation.GetDirectionVector(I, J, K)
strs.Add("")
strs.Add("Direction Vector: " + FormatVector(I, J, K))
End Sub

Private Sub CompositeDistanceBetweenData(ByVal annotation As
DimXpertCompositeDistanceBetweenDimTol)
    Dim feature As DimXpertFeature
    Dim featureFosUsage As Long
    Dim plus As Double, minus As Double
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    feature = Nothing
    ' the origin and tolerance feature along with their feature of size usage (min, max,
center)
    boolstatus = annotation.GetOriginFeature(feature, featureFosUsage)
    strs.Add("")
    strs.Add("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
    boolstatus = annotation.GetFeature(feature, featureFosUsage)
    strs.Add("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
    ' the pattern locating feature
    boolstatus = annotation.GetIntraFeature(feature, featureFosUsage)
    strs.Add("Pattern Locating Feature: " + feature.Name + " @ " +
FosUsage(featureFosUsage))
    ' The direction vector
    boolstatus = annotation.GetDirectionVector(I, J, K)
    strs.Add("")
    strs.Add("Direction Vector: " + FormatVector(I, J, K))
    ' the upper and lower tolerance value for the pattern location by type
    Dim dimTol As IDimXpertDimensionTolerance
    dimTol = annotation
    Select Case dimTol.DimensionType
        ' plus and minus dimension
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusNoNominal
            dimTol.GetPlusAndMinusTolerance(plus, minus)
            strs.Add("Pattern Locating Dimension Type: Plus Minus No Nominal")
            strs.Add(" Plus Tolerance: " + FormatDouble(plus))
            strs.Add(" Minus Tolerance: " + FormatDouble(minus))
        ' block tolerance

```

```

        Case
swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockToleranceNoNominal
            Dim blockTols As DimXpertBlockTolerances
            blockTols = dimXpertPart.GetBlockTolerances
            Select Case blockTols.Type
                Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ASMEInch
                    strs.Add("Pattern Locating Dimension Type: Block Tolerance No
Nominal")
                    strs.Add(" Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlaces, "##,##0"))
                Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ISO2768
                    strs.Add("Pattern Locating Dimension Type: General Tolerance")
            End Select
        End Select
    End Select
    ' the upper and lower tolerance value for the feature to feature location by type
    Select Case annotation.DimensionTypeIntraFeature
        ' plus and minus dimension
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusNoNominal
            strs.Add("Feature Locating Dimension Type: Plus Minus No Nominal")
            boolstatus = annotation.GetPlusAndMinusToleranceIntraFeature(plus, minus)
            strs.Add(" Plus Tolerance: " + FormatDouble(plus))
            strs.Add(" Minus Tolerance: " + FormatDouble(minus))
            ' block tolerance
        Case
swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockToleranceNoNominal
            Dim blockTols As DimXpertBlockTolerances
            blockTols = dimXpertPart.GetBlockTolerances
            Select Case blockTols.Type
                Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ASMEInch
                    strs.Add("Feature locating Dimension Type: Block Tolerance No
Nominal")
                    strs.Add(" Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlacesIntraFeature, "##,##0"))
                Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ISO2768
                    strs.Add("Feature locating Dimension Type: General Tolerance")
            End Select
        End Select
    End Select
End Sub
Private Sub DatumsStr(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Datums:")
    Call datumStr(annotation.GetPrimaryDatums(), annotation.GetPrimaryDatumModifiers(), "
Primary:")
    Call datumStr(annotation.GetSecondaryDatums(),
annotation.GetSecondaryDatumModifiers(), " Secondary:")
    Call datumStr(annotation.GetTertiaryDatums(), annotation.GetTertiaryDatumModifiers(),
" Tertiary:")
End Sub

```

```

Private Sub datumStr(ByVal dats As Object, ByVal mods As Object, ByVal datumOrder As
String)
    Dim I As Long
    Dim str As String
    Dim mcm As Long
    If IsNothing(dats) Then
        Exit Sub
    End If
    str = ""
    For I = LBound(dats) To UBound(dats)
        mcm = mods(I)
        str = str + " " + dats(I).Identifier + " @ " + mcmStr(mcm)
    Next I
    If StrComp(str, "") > 0 Then
        strs.Add(datumOrder + str)
    Else
        strs.Add(datumOrder + " <none>")
    End If
End Sub
' returns a string containing the height of the projected tolerance zone
Private Sub FormatProjectedZone(ByVal enabled As Boolean, ByVal height As Double)
    If enabled = True Then
        strs.Add(" Projected Zone: True")
        strs.Add(" Zone Height: " + FormatDouble(height))
    Else
        strs.Add(" Projected Zone: False")
    End If
End Sub
Private Function mcmStr(ByVal mcm As Long) As String
    Dim str As String
    str = ""
    Select Case mcm
        Case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_LMC
            str = "LMC"
        Case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_MMC
            str = "MMC"
        Case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_NoMCM
            str = "NoMCM"
        Case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_RFS
            str = "RFS"
    End Select
    mcmStr = str
End Function
' returns a string containing the type of position tolerance zone used
Private Function PositionZoneType(ByVal typ As Long) As String

```

```

Dim str As String
Select Case typ
    Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_CylindricalPosition
        str = "Cylindrical"
    Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_PlanarPosition
        str = "Planar"
    Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_SphericalPosition
        str = "Spherical"
    Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_Boundary
        str = "Boundary"
    Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_RadialPositionArc
        str = "RadialPositionArc"
    Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_RadialPositionPlanar
        str = "RadialPositionPlanar"
    Case Else
        str = "N/A"
End Select
PositionZoneType = str
End Function
' returns a string containing the names of the SW display entities
Private Function DisplayEntity(ByVal annotation As DimXpertAnnotation) As String
    Dim str As String
    Dim dispEnt As Object
    Dim swAnnot As SolidWorks.Interop.sldworks.Annotation
    str = ""
    'Set dispEnt = swDimXpert.GetDisplayEntity(annot)
    dispEnt = annotation.GetDisplayEntity()
    If Not dispEnt Is Nothing Then
        If TypeOf dispEnt Is SolidWorks.Interop.sldworks.Annotation Then
            swAnnot = dispEnt
            str = swAnnot.GetName
        End If
    End If
    DisplayEntity = str
End Function
' returns a string containing the feature of size usage (min, max, center)
Private Function FosUsage(ByVal value As Long) As String
    Dim str As String
    Select Case value
        Case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_Center
            str = "Center"
        Case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_MaximumSide
            str = "Max"
        Case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_MinimumSide
            str = "Min"
    End Select
End Function

```

```

        Case Else
            str = "N/A"
        End Select
        FosUsage = str
    End Function

    Private Function FormatVector(ByVal I As Double, ByVal J As Double, ByVal K As Double) As String
        Dim str As String
        str = FormatDouble(I) + ", " + FormatDouble(J) + ", " + FormatDouble(K)
        FormatVector = str
    End Function

    Private Function FormatDouble(ByVal value As Double) As String
        Dim str As String
        str = Format(value, "##,##0.0000")
        FormatDouble = str
    End Function

    Private Function RadiansToDegrees(ByVal value As Double) As String
        Dim str As String
        str = Format((value * 57.2957795130823), "##,##0.00")
        RadiansToDegrees = str
    End Function

    Private Function annotationTypeNameFromObject(ByVal anno As DimXpertAnnotation) As String
        annotationTypeNameFromObject = annotationTypeNameFromTypeNumber(anno.Type)
    End Function

    Private Function annotationTypeNameFromTypeNumber(ByVal annoTypeIndex As Long) As String
        Select Case annoTypeIndex
            Case swDimXpertAnnotationType_e.swDimXpertDimTol_DistanceBetween
                annotationTypeNameFromTypeNumber = "DistanceBetween Dim"
            Case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterBore
                annotationTypeNameFromTypeNumber = "CounterBore Dim"
            Case swDimXpertAnnotationType_e.swDimXpertDimTol_Depth
                annotationTypeNameFromTypeNumber = "Depth Dim"
            Case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkDiameter
                annotationTypeNameFromTypeNumber = "CounterSinkDiameter Dim"
            Case swDimXpertAnnotationType_e.swDimXpertDimTol_ChamferDimension
                annotationTypeNameFromTypeNumber = "ChamferDimension Dim"
            Case swDimXpertAnnotationType_e.swDimXpertDimTol_AngleBetween
                annotationTypeNameFromTypeNumber = "AngleBetween Dim"
            Case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkAngle
                annotationTypeNameFromTypeNumber = "CounterSinkAngle Dim"
            Case swDimXpertAnnotationType_e.swDimXpertDimTol_ConeAngle
                annotationTypeNameFromTypeNumber = "ConeAngle Dim"
            Case swDimXpertAnnotationType_e.swDimXpertDimTol_Diameter
                annotationTypeNameFromTypeNumber = "Diameter Dim"
            Case swDimXpertAnnotationType_e.swDimXpertDimTol_Length

```

```
        annotationTypeNameFromTypeNumber = "Length Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_Radius
        annotationTypeNameFromTypeNumber = "Radius Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_Width
        annotationTypeNameFromTypeNumber = "Width Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_CompositeDistanceBetween
        annotationTypeNameFromTypeNumber = "CompositeDistanceBetween Dim"
Case swDimXpertAnnotationType_e.swDimXpertDatum
        annotationTypeNameFromTypeNumber = "Datum"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Position
        annotationTypeNameFromTypeNumber = "Position Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositePosition
        annotationTypeNameFromTypeNumber = "CompositePosition Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Symmetry
        annotationTypeNameFromTypeNumber = "Symmetry Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Concentricity
        annotationTypeNameFromTypeNumber = "Concentricity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_LineProfile
        annotationTypeNameFromTypeNumber = "LineProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeLineProfile
        annotationTypeNameFromTypeNumber = "CompositeLineProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_SurfaceProfile
        annotationTypeNameFromTypeNumber = "SurfaceProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeSurfaceProfile
        annotationTypeNameFromTypeNumber = "CompositeSurfaceProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Angularity
        annotationTypeNameFromTypeNumber = "Angularity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Parallelism
        annotationTypeNameFromTypeNumber = "Parallelism Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Perpendicularity
        annotationTypeNameFromTypeNumber = "Perpendicularity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_TotalRunout
        annotationTypeNameFromTypeNumber = "TotalRunout Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CircularRunout
        annotationTypeNameFromTypeNumber = "CircularRunout Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Flatness
        annotationTypeNameFromTypeNumber = "Flatness Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Circularity
        annotationTypeNameFromTypeNumber = "Circularity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Cylindricity
        annotationTypeNameFromTypeNumber = "Cylindricity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Straightness
        annotationTypeNameFromTypeNumber = "Straightness Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Tangency
        annotationTypeNameFromTypeNumber = "Tangency Tol"
```

```
        Case Else
            annotationTypeNameFromTypeNumber = "<unknown> " & CStr(annoTypeIndex)
        End Select
    End Function
    Public swApp As SldWorks
End Class
```

Get_DimXpert_Width_And_BestFitPlane_Features_Example_VB (Topic)

SOLIDWORKS API Help

Get DimXpert Compound Width and Best Fit Plane Features Example (VBA)

This example shows how to build a part and get attributes for the following DimXpert features:

- * Compound width
- * Best fit plane

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\block.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Create the best fit plane feature:  
'   a. Click the Location Dimension icon on the DimXpert toolbar.  
'   b. Select the left front face of the block.  
'   c. Click the Compound Plane icon on the DimXpert pop up toolbar.  
'   d. Select the right front face of the block.  
'   e. Click the green check mark on the DimXpert pop up toolbar.  
'   f. Select the back face of the block.  
'   g. Click to place the location dimension annotation.  
' 4. Create the compound width feature:  
'   a. Click the Size Dimension icon on the DimXpert toolbar.  
'   b. Select a front face of the block.  
'   c. Click the Width icon on the DimXpert pop up toolbar.  
'   d. Select the back face of the block.  
'   e. Click the green check mark on the DimXpert pop up toolbar.  
'   f. Click to place the size dimension annotation.  
' 5. Observe the following DimXpert features on the DimXpertManager tab:  
'   Plane2, Plane3, Width1.  
' 6. Open an Immediate window.  
' 7. Ensure that the latest SOLIDWORKS DimXpert type library is loaded  
'   in Tools > References.  
'  
' Postconditions: Compare the output in the Immediate Window  
' with the features displayed on the DimXpertManager tab of the  
' Management Panel.  
'  
' NOTE: Because this part is used elsewhere, o not save changes.  
'-----
```

Option Explicit

Dim swApp As SldWorks.SldWorks

Dim swModel As ModelDoc2

Dim swModelDocExt As ModelDocExtension

Dim swSelMgr As SelectionMgr

Dim swConfig As Configuration

Dim swFeature As feature

Dim swAnn As feature

Dim swSchema As DimXpertManager

Dim swDXPart As DimXpertPart

Dim featureType As swDimXpertFeatureType

```

Dim features As Variant
Dim appliedFeatures As Variant
Dim appliedAnnotations As Variant
Dim appliedAnnotation As DimXpertAnnotation
Dim feature As DimXpertFeature
Dim appliedFeature As DimXpertFeature
Dim msgStr As String
Dim msgStr2 As String
Dim msgStr3 As String
Dim msgStr4 As String
Dim n As Long
Dim o As Long
Dim p As Long
Dim boolstatus As Boolean
Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    ' Get the default DimXpert schema using IModelDocExtension.DimXpertManager()

    Set swSchema = swModelDocExt.DimXpertManager("Default", True)

    ' Get IDimXpertPart from the IDimXpertManager
    Set swDXPart = swSchema.DimXpertPart

    Dim featCount As Long
    featCount = swDXPart.GetFeatureCount
    msgStr = "Total of "
    msgStr2 = featCount
    msgStr = msgStr + msgStr2 + " DimXpert features in " + (swSchema.SchemaName)
    Debug.Print ""
    Debug.Print msgStr

    ' Get IDimXpert features through IDimXpertPart

    features = swDXPart.GetFeatures

    msgStr = (swSchema.SchemaName) + " has the following features: "
    Debug.Print ""
    Debug.Print msgStr
    For n = 0 To UBound(features)
        Set feature = features(n)
        Debug.Print " " + "Feature name: " + (feature.Name)

        featureType = feature.Type
        Call GetPatternType(featureType, msgStr2)

        msgStr = "    Feature type "

```

```
msgStr3 = " is suppressed on the DimXpertManager tab? "
```

```
msgStr4 = feature.IsSuppressed()
```

```
Debug.Print msgStr + msgStr2 + msgStr3 + msgStr4
```

```
msgStr = " " + "Model feature: "
```

```
Set swFeature = feature.GetModelFeature()
```

```
If Not (swFeature Is Nothing) Then
```

```
    msgStr2 = swFeature.GetType2()
```

```
    Debug.Print msgStr + msgStr2
```

```
End If
```

```
msgStr = " " + "Number of SOLIDWORKS face entities in this feature: "
```

```
msgStr2 = feature.GetFaceCount
```

```
Debug.Print msgStr + msgStr2
```

```
msgStr = " " + "Number of applied features: "
```

```
msgStr2 = feature.GetAppliedFeatureCount()
```

```
Debug.Print msgStr + msgStr2
```

```
appliedFeatures = feature.GetAppliedFeatures()
```

```
If Not (IsEmpty(appliedFeatures)) Then
```

```
    For o = 0 To UBound(appliedFeatures)
```

```
        Set appliedFeature = appliedFeatures(o)
```

```
        Debug.Print " " + "Applied feature name: " + (appliedFeature.Name)
```

```
    Next
```

```
End If
```

```
msgStr = " " + "Number of applied annotations: "
```

```
msgStr2 = feature.GetAppliedAnnotationCount()
```

```
Debug.Print msgStr + msgStr2
```

```
appliedAnnotations = feature.GetAppliedAnnotations()
```

```
If Not (IsEmpty(appliedAnnotations)) Then
```

```
    For p = 0 To UBound(appliedAnnotations)
```

```
        Set appliedAnnotation = appliedAnnotations(p)
```

```
        Debug.Print " " + "Applied annotation name: " + (appliedAnnotation.Name)
```

```
    Next
```

```
End If
```

```
Debug.Print " "
```

```
Next
```

```
' If you know the name of a DimXpert feature, you can get it directly using IDimXpertPart.GetFeature("name"),
```

```
' which can return a general IDimXpertFeature or a more specific interface on the feature
```

```
' Get IDimXpertCompoundWidthFeature for the Width1 feature
```

```
Dim widthFeature As IDimXpertCompoundWidthFeature
```

```
Set widthFeature = swDXPart.GetFeature("Width1")
```

```
msgStr = widthFeature.Name + " is a DimXpert Width feature"
```

```
Debug.Print ""
```

```
Debug.Print msgStr
```

```
Debug.Print ""
```

```
' Get the nominal width coordinates
```

```
Dim width As Double
```

```
Dim x As Double
```

```
Dim y As Double
```

```
Dim z As Double
```

```
Dim i As Double
```

```
Dim j As Double
```

```
Dim k As Double
```

```
Debug.Print "Nominal width of Width1"
```

```
Debug.Print ""
```

```
boolstatus = widthFeature.GetNominalCompoundWidth(width, x, y, z, i, j, k)
```

```
msgStr = "Width is "
```

```
msgStr2 = width
```

```
Debug.Print msgStr + msgStr2
```

```
msgStr = "X-coordinate is "
```

```
msgStr2 = x
```

```
Debug.Print msgStr + msgStr2
```

```
msgStr = "Y-coordinate is "
```

```
msgStr2 = y
```

```
Debug.Print msgStr + msgStr2
```

```
msgStr = "Z-coordinate is "
```

```
msgStr2 = z
```

```
Debug.Print msgStr + msgStr2
```

```
msgStr = "I-component of pierce vector is "
```

```
msgStr2 = i
```

```
Debug.Print msgStr + msgStr2
```

```
msgStr = "J-component of pierce vector is "
```

```
msgStr2 = j
```

```
Debug.Print msgStr + msgStr2
```

```
msgStr = "K-component of pierce vector is "
```

```
msgStr2 = k
```

```
Debug.Print msgStr + msgStr2
```

```
Debug.Print ""
```

```
' Get whether the width is a hole or a pin
```

```
boolstatus = widthFeature.Inner
```

```
msgStr = "The width is for a hole and not a pin: "
```

```
msgStr2 = boolstatus
```

```
Debug.Print msgStr + msgStr2
```

```
' Get IDimXpertBestfitPlaneFeature for the Plane2 feature
```

```
Dim bestfitPlaneFeature As IDimXpertBestfitPlaneFeature
```

```
Set bestfitPlaneFeature = swDXPart.GetFeature("Plane2")
```

```
msgStr = bestfitPlaneFeature.Name + " is a DimXpert Bestfit Plane feature"
```

```
Debug.Print ""
Debug.Print msgStr
Debug.Print ""
```

```
Dim featureCount As Integer
featureCount = bestfitPlaneFeature.GetSubFeatureCount
msgStr = "The number of sub-features of the bestfit plane is "
msgStr2 = featureCount
Debug.Print msgStr + msgStr2
```

```
features = bestfitPlaneFeature.GetSubFeatures
For n = 0 To UBound(features)
    Set feature = features(n)
    Debug.Print " " + "Feature name: " + (feature.Name)
```

```
    featureType = feature.Type
    Call GetPatternType(featureType, msgStr2)
```

```
    msgStr = "    Feature type is "
```

```
    Debug.Print msgStr + msgStr2
Next
```

```
End Sub
```

```
Public Sub GetPatternType(ByRef featureType, ByRef msgStr2)
    If (featureType = swDimXpertFeature_Plane) Then
        msgStr2 = "Plane"
    ElseIf (featureType = swDimXpertFeature_Cylinder) Then
        msgStr2 = "Cylinder"
    ElseIf (featureType = swDimXpertFeature_Cone) Then
        msgStr2 = "Cone"
    ElseIf (featureType = swDimXpertFeature_Extrude) Then
        msgStr2 = "Extrude"
    ElseIf (featureType = swDimXpertFeature_Fillet) Then
        msgStr2 = "Fillet"
    ElseIf (featureType = swDimXpertFeature_Chamfer) Then
        msgStr2 = "Chamfer"
    ElseIf (featureType = swDimXpertFeature_CompoundHole) Then
        msgStr2 = "CompoundHole"
    ElseIf (featureType = swDimXpertFeature_CompoundWidth) Then
        msgStr2 = "CompoundWidth"
    ElseIf (featureType = swDimXpertFeature_CompoundNotch) Then
        msgStr2 = "CompoundNotch"
    ElseIf (featureType = swDimXpertFeature_CompoundClosedSlot3D) Then
        msgStr2 = "CompoundClosedSlot3D"
    ElseIf (featureType = swDimXpertFeature_IntersectPoint) Then
        msgStr2 = "IntersectPoint"
    ElseIf (featureType = swDimXpertFeature_IntersectLine) Then
        msgStr2 = "IntersectLine"
    ElseIf (featureType = swDimXpertFeature_IntersectCircle) Then
        msgStr2 = "IntersectCircle"
```

```
ElseIf (featureType = swDimXpertFeature_IntersectPlane) Then
    msgStr2 = "IntersectPlane"
ElseIf (featureType = swDimXpertFeature_Pattern) Then
    msgStr2 = "Pattern"
ElseIf (featureType = swDimXpertFeature_Sphere) Then
    msgStr2 = "Sphere"
ElseIf (featureType = swDimXpertFeature_BestfitPlane) Then
    msgStr2 = "Bestfit Plane"
ElseIf (featureType = swDimXpertFeature_Surface) Then
    msgStr2 = "Surface"
End If

End Sub
```

Get DimXpert Compound Width and Best Fit Plane Features Example (VB.NET)

This example shows how to build a part and get attributes for the following DimXpert features:

- * Compound width
- * Best fit plane

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\block.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Create the best fit plane feature:  
'   a. Click the Location Dimension icon on the DimXpert toolbar.  
'   b. Select the left front face of the block.  
'   c. Click the Compound Plane icon on the DimXpert pop up toolbar.  
'   d. Select the right front face of the block.  
'   e. Click the green check mark on the DimXpert pop up toolbar.  
'   f. Select the back face of the block.  
'   g. Click to place the location dimension annotation.  
' 4. Create the compound width Feature:  
'   a. Click the Size Dimension icon on the DimXpert toolbar.  
'   b. Select a front face of the block.  
'   c. Click the Width icon on the DimXpert pop up toolbar.  
'   d. Select the back face of the block.  
'   e. Click the green check mark on the DimXpert pop up toolbar.  
'   f. Click to place the size dimension annotation.  
' 5. Observe the following DimXpert features on the DimXpertManager tab:  
'   Plane2, Plane3, Width1.  
' 6. Open an Immediate window.  
' 7. Ensure that the SolidWorks.Interop.swdimxpert.dll interop assembly  
'   is loaded (right-click on the project in the Project Explorer  
'   and click Add Reference > .NET tab).  
'  
' Postconditions: Compare the output in the Immediate window  
' with the features displayed on the DimXpertManager tab of the  
' Management Panel.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swdimxpert  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics  
Partial Class SolidWorksMacro
```

```
    Dim swModel As IModelDoc2  
    Dim swModelDocExt As IModelDocExtension  
    Dim swSelMgr As ISelectionMgr
```

```

Dim swConfig As IConfiguration
Dim swFeature As IFeature
Dim swAnn As IFeature
Dim swSchema As IDimXpertManager
Dim swDXPart As IDimXpertPart
Dim featureType As swDimXpertFeatureType_e
Dim features As Object
Dim appliedFeatures As Object
Dim appliedAnnotations As Object
Dim appliedAnnotation As IDimXpertAnnotation
Dim feature As IDimXpertFeature
Dim appliedFeature As IDimXpertFeature
Dim msgStr As String
Dim msgStr2 As String
Dim msgStr3 As String
Dim msgStr4 As String
Dim n As Long
Dim o As Long
Dim p As Long
Dim boolstatus As Boolean
Sub main()
    swModel = swApp.ActiveDoc
    swModelDocExt = swModel.Extension
    swSelMgr = swModel.SelectionManager
    ' Get the default DimXpert schema using IModelDocExtension.DimXpertManager()
    swSchema = swModelDocExt.DimXpertManager("Default", True)
    ' Get IDimXpertPart from the IDimXpertManager
    swDXPart = swSchema.DimXpertPart
    Dim featCount As Long
    featCount = swDXPart.GetFeatureCount
    msgStr = "Total of "
    msgStr2 = featCount
    msgStr = msgStr + msgStr2 + " DimXpert features in " + (swSchema.SchemaName)
    Debug.Print("")
    Debug.Print(msgStr)
    ' Get IDimXpert features through IDimXpertPart
    features = swDXPart.GetFeatures
    msgStr = (swSchema.SchemaName) + " has the following features: "
    Debug.Print("")
    Debug.Print(msgStr)
    For n = 0 To UBound(features)
        feature = features(n)
        Debug.Print(" " + "Feature name: " + (feature.Name))
        featureType = feature.Type
        Call GetPatternType(featureType, msgStr2)
        msgStr = "   Feature type "
        msgStr3 = " is suppressed on the DimXpertManager tab? "
        msgStr4 = feature.IsSuppressed()
        Debug.Print(msgStr + msgStr2 + msgStr3 + msgStr4)
        msgStr = "   " + "Model feature: "
        swFeature = feature.GetModelFeature()
    
```

```

If Not (swFeature Is Nothing) Then
    msgStr2 = swFeature.GetTypeName2()
    Debug.Print(msgStr + msgStr2)
End If

msgStr = "    " + "Number of SOLIDWORKS face entities in this feature: "
msgStr2 = feature.GetFaceCount
Debug.Print(msgStr + msgStr2)

msgStr = "    " + "Number of applied features: "
msgStr2 = feature.GetAppliedFeatureCount()
Debug.Print(msgStr + msgStr2)
appliedFeatures = feature.GetAppliedFeatures()
If Not (IsNothing(appliedFeatures)) Then
    For o = 0 To UBound(appliedFeatures)
        appliedFeature = appliedFeatures(o)
        Debug.Print("        " + "Applied feature name: " + (appliedFeature.Name))
    Next
End If

msgStr = "    " + "Number of applied annotations: "
msgStr2 = feature.GetAppliedAnnotationCount()
Debug.Print(msgStr + msgStr2)
appliedAnnotations = feature.GetAppliedAnnotations()
If Not (IsNothing(appliedAnnotations)) Then
    For p = 0 To UBound(appliedAnnotations)
        appliedAnnotation = appliedAnnotations(p)
        Debug.Print("        " + "Applied annotation name: " + (appliedAnnotation.Name))
    Next
End If
Debug.Print("    ")
Next

' If you know the name of a DimXpert feature, you can get it directly using IDimXpertPart.GetFeature("name"),
' which can return a general IDimXpertFeature or a more specific interface on the feature
' Get IDimXpertCompoundWidthFeature for the Width1 feature
Dim widthFeature As IDimXpertCompoundWidthFeature
widthFeature = swDXPart.GetFeature("Width1")
msgStr = widthFeature.Name + " is a DimXpert Width feature"
Debug.Print("")
Debug.Print(msgStr)
Debug.Print("")
' Get the nominal width coordinates
Dim width As Double
Dim x As Double
Dim y As Double
Dim z As Double
Dim i As Double
Dim j As Double
Dim k As Double
Debug.Print("Nominal width of Width1")
Debug.Print("")
boolstatus = widthFeature.GetNominalCompoundWidth(width, x, y, z, i, j, k)
msgStr = "Width is "
msgStr2 = width

```

```

Debug.Print(msgStr + msgStr2)
msgStr = "X-coordinate is "
msgStr2 = x
Debug.Print(msgStr + msgStr2)
msgStr = "Y-coordinate is "
msgStr2 = y
Debug.Print(msgStr + msgStr2)
msgStr = "Z-coordinate is "
msgStr2 = z
Debug.Print(msgStr + msgStr2)
msgStr = "I-component of pierce vector is "
msgStr2 = i
Debug.Print(msgStr + msgStr2)
msgStr = "J-component of pierce vector is "
msgStr2 = j
Debug.Print(msgStr + msgStr2)
msgStr = "K-component of pierce vector is "
msgStr2 = k
Debug.Print(msgStr + msgStr2)
Debug.Print("")
' Get whether the width is a hole or a pin
boolstatus = widthFeature.Inner
msgStr = "The width is for a hole and not a pin: "
msgStr2 = boolstatus
Debug.Print(msgStr + msgStr2)
' Get IDimXpertBestfitPlaneFeature for the Plane2 feature
Dim bestfitPlaneFeature As IDimXpertBestfitPlaneFeature
bestfitPlaneFeature = swDXPart.GetFeature("Plane2")
msgStr = bestfitPlaneFeature.Name + " is a DimXpert Bestfit Plane feature"
Debug.Print("")
Debug.Print(msgStr)
Debug.Print("")
Dim featureCount As Integer
featureCount = bestfitPlaneFeature.GetSubFeatureCount
msgStr = "The number of sub-features of the bestfit plane is "
msgStr2 = featureCount
Debug.Print(msgStr + msgStr2)
features = bestfitPlaneFeature.GetSubFeatures
For n = 0 To UBound(features)
    feature = features(n)
    Debug.Print(" " + "Feature name: " + (feature.Name))
    featureType = feature.Type
    Call GetPatternType(featureType, msgStr2)
    msgStr = "    Feature type is "
    Debug.Print(msgStr + msgStr2)
Next
End Sub

Public Sub GetPatternType(ByRef featureType As swDimXpertFeatureType_e, ByRef msgStr2 As String)
    If (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Plane) Then
        msgStr2 = "Plane"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Cylinder) Then

```

```

    msgStr2 = "Cylinder"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Cone) Then
    msgStr2 = "Cone"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Extrude) Then
    msgStr2 = "Extrude"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Fillet) Then
    msgStr2 = "Fillet"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Chamfer) Then
    msgStr2 = "Chamfer"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole) Then
    msgStr2 = "CompoundHole"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth) Then
    msgStr2 = "CompoundWidth"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch) Then
    msgStr2 = "CompoundNotch"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D) Then
    msgStr2 = "CompoundClosedSlot3D"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint) Then
    msgStr2 = "IntersectPoint"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine) Then
    msgStr2 = "IntersectLine"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle) Then
    msgStr2 = "IntersectCircle"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane) Then
    msgStr2 = "IntersectPlane"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Pattern) Then
    msgStr2 = "Pattern"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Sphere) Then
    msgStr2 = "Sphere"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane) Then
    msgStr2 = "Bestfit Plane"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Surface) Then
    msgStr2 = "Surface"
End If
End Sub

```

```

Public swApp As SldWorks
End Class

```

SolidWorks.Interop.swdimxpert (Topic)

SOLIDWORKS DimXpert API Help

SolidWorks.Interop.swdimxpert Assembly

SOLIDWORKS DimXpert API

Namespaces

Namespace	Description
SolidWorks.Interop.swdimxpert	SOLIDWORKS DimXpert API

[Send comments on this topic.](#)